

Highlights

From a Production OTP Flood to a Layered Defence: Measured Limits of Risk-Scored SMS Verification Against Residential Flooders and Colluding Carriers

Naveed Ahmed

- A production OTP flood, the first fix, and how the attacker adapted to it
- Per-request-recognisable attacks are stopped with 99 % of real users delivered
- A human-like residential flooder leaves only rationing; the trade-off is measured
- Pumper leakage is about 5 messages per destination block touched, plus in-flight
- The block tests' false-positive cost is measured over 24 h and a carrier outage

From a Production OTP Flood to a Layered Defence: Measured Limits of Risk-Scored SMS Verification Against Residential Flooders and Colluding Carriers

Naveed Ahmed^{a,*}

^a*TODO: affiliation, TODO, TODO, Saudi Arabia*

Abstract

A registration endpoint that sends a one-time password (OTP) by SMS to any number it is given will be used to send SMS at someone else's expense. We describe such an incident, the first set of controls that stopped it, and how the attacker adapted to them by rotating addresses, spoofing client attributes and staying under every per-key threshold. We then present a twelve-step pipeline that replaces claimed trust with earned trust: platform exemptions require app attestation, every client carries a signed identity, numbers are checked for cost and liveness before any money is spent, and a risk score built on whether earlier codes were ever verified decides between sending, delaying, challenging, moving to a cheaper channel or refusing. The pipeline is implemented and evaluated in a calibrated simulation with legitimate traffic in the background, thirty seeds per attacker and confidence intervals, since no production logs were available. Attacks that can be recognised per request are stopped completely while 99 % of real users are delivered. A residential attacker with human-like CAPTCHA scores and valid numbers is not separated by any behavioural signal at any volume tested; the only containment is rationing, which holds it to 241 of 600 messages by refusing 42 % of first-time users. Against SMS pumping the destination number is the one key the attacker cannot rotate, and a sequential test on 10 000-number blocks bounds leakage to about five messages per block plus whatever is in flight; a colluding carrier evades it only by verifying 42 % of its codes, each a fake account. The tests' cost to real users is measured over 24 hours and a carrier

*Corresponding author.

Email address: `optimistbs@gmail.com` (Naveed Ahmed)

outage. Code, evaluation and results are public.

Keywords: SMS pumping, artificially inflated traffic, one-time password, rate limiting, risk scoring, residential proxies, fraud prevention, authentication

1. Introduction

Most online services verify a phone number the same way: the user types the number, the service sends it a code by SMS, the user types the code back. The sending step costs money, between a tenth of a cent and twenty cents per message depending on the destination, and the endpoint that triggers it is reachable before any account exists. That combination makes it a target in two ways. A flooder can trigger thousands of messages to numbers it does not own, which costs the service money and its users delivery. A pumper does the same to numbers on a range whose operator shares the termination fee with it, which turns the service's SMS budget into the attacker's income. The second practice is called SMS pumping or artificially inflated traffic (AIT), and MITRE lists it as a resource-hijacking technique [1]. The scale is not small: in 2023 Twitter's owner stated publicly that the company had been losing about 60 million dollars a year to telecommunications operators that used bot accounts to pump two-factor SMS [2], and Huh et al. report that the 3-month log of one large authentication service contained as many attack requests as genuine ones in the nineteen countries they studied [3].

This paper starts from such an incident on a registration flow we operated. The first response was the usual one: a hidden CAPTCHA, one message per number per minute, five requests per IP per minute, a country allow-list, a limit on message length, and a block on VPN and proxy connections. It stopped the attack it was built for. The attack that came back a few weeks later was different: it rotated through residential addresses, sent the header that marked a request as coming from the iOS app, and kept every number and every address under its threshold. Reviewing the first design against this attacker exposed eight gaps, and the second design, which this paper describes, closes them.

The second design is a pipeline of twelve steps and an asynchronous feedback loop. Its organising idea is that trust should be earned by behaviour, not claimed in a header. Platform exemptions are granted only after Apple App Attest or Google Play Integrity has verified the app. Every client carries

32 a signed session token bound to a device fingerprint, so an identity survives
33 an address change. Numbers are checked for country, prefix cost class, gen-
34 erated patterns and liveness before a message is paid for. Each network,
35 device, country and number range carries a reputation derived from one fact
36 the service can observe and the attacker cannot forge without owning phones:
37 whether the codes it asked for were ever entered. A risk score built on these
38 signals chooses among five responses, from sending at once to refusing, and a
39 global circuit breaker bounds the total spend whatever the traffic looks like.

40 We built it, and then we tried to break it. The evaluation is a simulation,
41 because no production logs from the incident were available to us, and we
42 say so wherever a number depends on that. Within it we ask five questions.

43 RQ1 Which attacks does the design stop, compared with the first one, and
44 at what cost to real users?

45 RQ2 Which layer is responsible for stopping each attack?

46 RQ3 When does reputation built on verification outcomes separate an at-
47 tacker from real users, and when does it not?

48 RQ4 When it does not, what does containment cost?

49 RQ5 What do the tests that act on a shared key, the destination number
50 block, cost real users when the population differs from the calibration
51 or a carrier fails?

52 The answers are not all good news for the design, and the paper is writ-
53 ten around that. Three results carry it. Attacks that can be recognised
54 per request, a spoofed platform header, datacenter rotation, a premium-rate
55 range, a single client, a sequential walk through a number range, are stopped
56 completely with 99 % of real users still delivered. A residential attacker that
57 looks like a real user leaves nothing but rationing, and the paper measures
58 the trade-off between what it leaks and how many real users are refused. SMS
59 pumping is separable on the destination block, with a closed-form bound on
60 what it costs and a measured false-positive price, until the colluding carrier
61 verifies enough of its own codes, at which point the cost moves downstream
62 into fake accounts.

63 The contributions are:

- 64 1. A documented case of an OTP flood, the controls that stopped it, and
65 the adapted attack that defeated them, as eight named gaps (Section 3).

2. A layered design that closes those gaps with graded responses and explicit limits on collateral damage, with an open implementation (Sections 4 and 6).
3. A calibrated, seeded evaluation against nine attacker profiles, six adaptive attackers and four pumpers, with an ablation on the same seeds (Section 7).
4. A separability result: outcome-based reputation separates an attacker only on keys the attacker dominates, shown by a dilution curve and a parameter sweep that cannot move it (Section 7.3).
5. A closed-form model of pumping leakage against destination-block tests, checked against simulation, with its evasion threshold and its false-positive cost (Sections 5, 7.5 and 7.6).

The rest of the paper follows this order: related work (Section 2), the incident and threat model (Section 3), the design (Section 4), the model (Section 5), the evaluation method (Section 6), results by question (Section 7), lessons and limitations (Section 8) and a conclusion.

2. Background and related work

2.1. *Where the money goes*

Telephony fraud is old, and most of its forms depend on the gap between what a call or message costs to deliver and what someone is paid for terminating it. Sahin et al. organised the field in a taxonomy of root causes, weaknesses, techniques and schemes, and placed international revenue share fraud (IRSF) among the revenue-share schemes: a party advertises a range of numbers as premium-rate destinations, traffic is generated to them, and the termination fee is split along the route [4]. They note that the causes, legacy infrastructure, opaque routing where each operator knows only the next hop, and operators' reluctance to share pricing or fraud findings, are unlikely to be fixed soon. SMS is a minor topic in that survey; its mechanics are those of voice.

In later work Sahin and Francillon collected 3.14 million test numbers advertised by twelve IRSF providers over four years and showed that the number range is the attacker's managed asset: about 80 % of ranges were shared across providers, 57 % were mobile ranges, and a single block of a thousand Latvian numbers was carved into sub-ranges by six providers in one year [5]. They also showed why blocking by range is dangerous. Coarse

101 blocking of frequent IRSF destinations has made whole countries unreach-
 102 able, and their own naive range-based detector had a 5.8 % false-positive
 103 rate against 0.26 % for a classifier trained on call features. Two things carry
 104 over to SMS pumping: the destination range is what the fraudster cannot
 105 easily change, and anything that acts on a range must account for the real
 106 subscribers in it. Our destination-block tests (Section 4.4) and their false-
 107 positive study (Section 7.6) are built on exactly that pair.

108 *2.2. Detecting artificially inflated SMS traffic*

109 The closest work to ours is by Huh et al. [3]. They had what we did not:
 110 9.4 million SMS authentication logs from a service with tens of millions of
 111 users, from nineteen countries over three months in 2023, labelled by a peak-
 112 traffic heuristic into 4.95 million attack and 4.50 million genuine requests.
 113 They trained gradient-boosted trees on three kinds of features, per request,
 114 per user or device over 24 hours, and country-wide, and reached a recall of
 115 89.6 % at a false-positive rate of 0.19 % on web traffic and 91.1 % at 0.11 %
 116 on native traffic. The system is deployed and blocks a request when its score
 117 passes a threshold.

118 Three of their findings shaped our design. First, per-entity rate limits do
 119 not work against this attacker: about 73 % of web attacks sent four or fewer
 120 requests per identity in 24 hours, below any limit a service would set. Second,
 121 the strongest signals were not the client’s but the destination’s: the number
 122 of distinct numbers sharing a prefix in a day was the dominant native feature,
 123 and attacks showed about 92 % zero conversion where genuine traffic showed
 124 21 %. Third, their evasion analysis found that an attacker would need about
 125 48 times as many phone prefixes to evade the web model.

126 Where we differ is in what the system does and what it needs. Theirs
 127 is a classifier that needs labelled traffic and answers block or allow; ours is
 128 a prevention pipeline that works from day one without labels, acts online
 129 with a graded response, and carries explicit bounds on the harm a wrong
 130 decision does. We also treat their main signal, verification outcome per key,
 131 as the central object of study rather than one feature among many, and ask
 132 where it stops working. We see the two as complementary: a trained model
 133 of their kind would be one more input to our risk score, not a replacement for
 134 the limits around it. We note that their labels came from peak periods and
 135 that they report low recall on non-peak attacks; our low-and-slow attackers
 136 (Section 7.7) are the case their labelling would have missed.

137 *2.3. Defences in practice and their known limits*

138 Commercial SMS verification services ship a fraud filter. Twilio’s Ver-
139 ify Fraud Guard, the documented example, blocks verifications to regions
140 and countries known for pumping, flags destinations that are new relative
141 to an account’s history, and offers three protection levels with stated false-
142 positive rates under 1 % and under 2 % [6]. It is a destination-side filter at
143 the provider; it has no view of the client, the session or the fingerprint, and
144 it is as good as the provider’s list.

145 The client-side controls that services deploy first are a CAPTCHA and
146 an IP limit, and both have measured limits. Motoyama et al. showed in 2010
147 that CAPTCHA solving is a labour market, with retail prices of a dollar per
148 thousand, 86 to 89 % accuracy and median response times of ten to twenty
149 seconds, and argued that a CAPTCHA should be judged as friction on the
150 attacker’s economics rather than as a barrier [7]. Dionysiou and Athana-
151 sopoulos later classified the automated solvers and concluded that machine
152 learning alone was not sufficient to break text CAPTCHAs, while the human-
153 labour route remained open [8]. We price the solving step at 0.3 cents per
154 token in our threat model, from current market prices, and the evaluation
155 gives the attacker farmed scores indistinguishable from humans. On the IP
156 side, Mi et al. infiltrated five residential proxy services and found 6.18 million
157 addresses, 95 % of them residential, 90 % relaying for under fifteen minutes
158 before rotating, and only 2.2 % on any blacklist, with an average of 22 days
159 before one was listed [9]. Richter et al. measured that more than 90 % of
160 cellular networks sit behind carrier-grade NAT, with up to 128 subscribers
161 behind one public address, and concluded that address-based reputation and
162 blocking are “all but infeasible” there [10]. Our design treats the address
163 as one weak signal among several and gives carrier NAT ranges a higher
164 per-address allowance; the evaluation measures what happens when it does
165 not.

166 *2.4. Risk-based decisions and the security of SMS codes*

167 Risk-based authentication scores a login from implicit features and asks
168 for more proof when the score is high. Wiefling et al. probed eight large
169 services with 224 accounts and found that every service that applied risk-
170 based authentication used the IP address, that most added the user agent
171 or screen resolution, and that the ones with a single feature re-authenticated
172 users often [11]. Our Step 7 is a risk engine of that kind applied before
173 a message is sent rather than after a password, with the difference that

174 its strongest signal, the verification history of each key, is produced by the
175 system’s own earlier decisions.

176 The security of the code once sent is a separate problem and we do not
177 address it. Mulliner et al. showed in 2013 that smartphone trojans intercept
178 SMS codes and proposed a dedicated channel as a defence [12]; Siadati et al.
179 showed in this journal that users can be talked into forwarding a code and
180 that rewording the message cut the attack’s success from 50 % to 8 % [13].
181 Those attacks target the user who asked for a code. Ours targets the service
182 that sends it.

183 2.5. *What was missing*

184 Taken together, the literature establishes that the attacker can rotate ad-
185 dresses, fingerprints and CAPTCHA solutions cheaply, that the destination
186 range is what a pumper cannot rotate, and that acting on a range harms real
187 subscribers. What it does not contain is a measurement of where behaviour-
188 based reputation stops separating attackers from users, nor a false-positive
189 accounting for reputation keyed on the destination. That is what this paper
190 adds.

191 3. The incident and the adapted attacker

192 3.1. *What happened*

193 The registration flow sent an SMS to any unregistered number it was
194 given. During its first weeks the endpoint was called repeatedly with ran-
195 domly generated numbers in the allowed country, and every call produced
196 a paid message. There was no limit per number, per address or in total,
197 no check on the message body, no restriction on country, and nothing that
198 distinguished a browser from a script.

199 The first set of controls, which we call v1, is listed in Table 1. It was
200 conventional and it worked against the attack it was built for: a hidden
201 reCAPTCHA v3 with a score threshold on web requests, one message per
202 number per minute, five requests per address per minute, a country allow-
203 list with one country exempted for app users, a 420-character limit on the
204 body, a block on connections that a proxy-detection service flagged, and
205 per-platform, per-country caps tuned to historical peaks.

Table 1: The v1 controls and the gaps the adapted attacker exposed in them.

Gap	v1 assumed	How it was bypassed
A	The <code>HTTP_PLATFORM</code> header says which platform sent the request; iOS skips the host check, apps skip reCAPTCHA	Sending the header. Every platform-based exemption was reachable by anyone
B	Five requests per address per minute bounds one attacker	Residential proxy pools give a fresh address per request; nothing keyed the attacker across addresses
C	A number passed to the provider is worth sending to	Random and sequential numbers cost the same as real ones; nothing checked liveness
D	An allowed country is a safe destination	Premium-rate and revenue-share ranges inside the country cost many times a standard message
E	Per-key caps bound total spend	A wide attack under every cap burned budget without bound; nothing watched the total
F	Check-then-increment is a rate limit	Two instances racing both passed; under a flood the race was hit constantly
G	A bulk flag and a test-server flag are internal	Neither required stronger authentication than a normal request
H	Each check is pass or fail and the response says which failed	Thresholds were probed one at a time; registered numbers could be enumerated

206 3.2. How the attacker adapted

207 The abuse that returned did not trip v1. Requests arrived from thou-
208 sands of addresses in residential ranges, each used a handful of times. They
209 carried the iOS platform header and so skipped both the CAPTCHA and the
210 host check. They kept each number under one message a minute and each
211 address under five a minute. Reviewing v1 against this attacker produced
212 the eight gaps in Table 1. They are not implementation bugs; each one is an
213 assumption that held for the first attacker and not for the second.

214 3.3. Threat model

215 We assume an attacker who wants either to make the service send as many
216 messages as possible (a flooder) or to make it send messages to numbers on

217 which the attacker is paid (a pumper). The attacker can buy residential
218 proxy access at about 3 USD per gigabyte, a fresh browser fingerprint at no
219 cost, and reCAPTCHA v3 tokens with human-like scores at about 0.3 cents
220 each; can generate numbers at will; and knows the service’s thresholds, since
221 it can probe them. A pumper additionally controls, through a colluding
222 operator or reseller, a set of destination ranges, and may be able to read
223 the messages delivered to them and enter the codes. Interactive challenges
224 can be bought solved. The attacker cannot forge an app attestation, cannot
225 change the number ranges its partner terminates, and cannot make a real
226 person on an unrelated number verify a code. These capabilities and prices
227 are the ones the evaluation uses; Table 4 lists the profiles built from them.

228 4. Design

229 Two principles drive the design. Cheap local checks run first, so that obvi-
230 ous abuse is refused before a network call to a CAPTCHA or number-lookup
231 vendor and long before a message is paid for. Trust is earned by behaviour
232 and never claimed by a header: platform exemptions require attestation,
233 clients need a signed session, and every key a request touches carries a rep-
234 utation derived from whether the codes it asked for were ever verified.

235 4.1. The pipeline

236 Every OTP request passes through twelve steps (Table 2); the first failing
237 step stops it, and whatever the outcome the client receives the same response.
238 Steps 0 to 6 are hard rejects on cheap or local evidence. Step 7 turns the
239 evidence collected so far into a score and a tier. Steps 8 to 10 are the capacity
240 controls. Step 11 picks a channel, writes the audit record and answers.

241 Three steps deserve a word because the evaluation turns on them. Step 0
242 never reads the platform from the header. An iOS request is one with a valid
243 App Attest assertion, an Android request one with a valid Play Integrity
244 verdict; an invalid attestation is a stronger signal of abuse than none and
245 is blocked; a request with neither is a web request and gets no exemption.
246 Step 2 makes every rate limit a single Lua script in Redis, so that check and
247 increment are one operation across all instances. Step 11 returns HTTP 200
248 with the same body for every outcome from Step 1 onward and pads the
249 handler to a 400 ms floor, so that neither the body nor the timing tells the
250 caller which step refused it.

251 4.2. The risk score and its tiers

252 Step 7 computes

$$s = \sum_i w_i x_i + w_c \frac{\max(0, \theta - r_{\min})}{\theta} - w_v v_n - w_f v_f, \quad (1)$$

253 where the x_i are the per-request signals of Table 3, v_n and v_f indicate a
 254 number and a fingerprint with verified history, $r_{\min}$ is the lowest conversion
 255 ratio over the keys the request touches (address, subnet, ASN, fingerprint,
 256 country, prefix and destination block), $\theta = 0.3$ and $w_c = 25$. The score is
 257 clamped to $[0, 100]$ and mapped to a tier at 20, 40, 60 and 80: *allow* sends at
 258 once; *delay* sends after a few seconds that double per request in the session;
 259 *challenge* returns an interactive challenge to a browser and re-enters Step 7
 260 with a -20 credit when it is solved (apps are downgraded instead); *downgrade*
 261 may use push, WhatsApp or silent network authentication but never SMS;
 262 *block* refuses. In the circuit breaker’s elevated mode every boundary moves
 263 down by ten points; in emergency mode only a score under 10 may use SMS.

264 No single signal blocks. The largest contribution a shared key can make,
 265 the conversion penalty, moves a clean-looking web client only to *delay*, which
 266 still sends; that is deliberate, since the penalty attaches to keys such as a
 267 country or a residential ASN that real users share with whoever is attacking
 268 through them. Refusing needs signals to stack, and a distributed attacker
 269 supplies them by construction: a rotating client has a fresh fingerprint, a
 270 reused one accumulates its own history, generated numbers trip the pattern
 271 detectors.

272 4.3. The feedback loop

273 Each send is attached to the reputation keys of its request. When the
 274 code is entered, every key’s *verified* counter rises; when five wrong attempts
 275 are made, or two minutes pass after delivery with no attempt, *failed* rises;
 276 a verification that arrives after that is reclassified. Counters live in hourly
 277 buckets over a rolling day. The ratio *verified* / (*verified* + *failed*) counts re-
 278 solved sends only, so a burst of fresh legitimate traffic, a campaign launch
 279 for instance, is neutral until its codes have had time to be entered. The
 280 two-minute window replaced the ten-minute code validity after we measured
 281 that 99 % of simulated real users verify within about 100 seconds; waiting
 282 the full validity only delayed the signal.

283 Two things gate what counts. A send counts as failed only after the
284 provider’s delivery receipt confirmed it arrived; a send with no receipt inside
285 sixty seconds, or a failed one, is *undelivered* and feeds neither the ratio nor the
286 block tests. And verification speed, used below, is clocked from the receipt,
287 not from the send, so that a slow route cannot make a person look like a
288 machine. A provider without delivery reports is configured to clock from the
289 send, as our first version did.

290 Fine keys whose ratio falls below 0.1 over at least fifty resolved sends are
291 placed on a 24-hour denylist: addresses, subnets, fingerprints and hosting
292 ASNs. A residential ASN is never denylisted, because it holds thousands of
293 real people and denylisting it would hand the attacker a denial of service;
294 a residential attack is handled by the score instead, where the conversion
295 penalty and the sustained-flood bonus move every request on that key into
296 the challenge tier, so that real users on the same ISP solve a challenge and
297 are served while the bot is not.

298 4.4. *Destination-block tests*

299 The keys above are all ones the attacker can rotate. The destination is the
300 one a pumper cannot: it is paid only on the numbers its partner terminates.
301 We therefore key reputation also on the 8-digit block of the destination,
302 10 000 numbers, and judge blocks by sequential probability-ratio tests [14]
303 rather than by the shared-key thresholds, because a block is touched by
304 legitimate traffic a fraction of a time per day and a block with several sends
305 is almost certainly one party’s.

306 Two tests run per block, each reaching a verdict when the likelihood ra-
307 tio of attacker to legitimate behaviour exceeds 1 000. The conversion test
308 assumes real users verify 80 % of codes and a flooder at most 10 %: five
309 unverified sends in a row give a ratio of $(0.9/0.2)^5 = 1\,845$, and a real pop-
310 ulation at 80 % conversion produces that with probability 0.2^5 , three in ten
311 thousand. The speed test assumes a machine enters a code within five sec-
312 onds of delivery 90 % of the time; real users do so whenever their operating
313 system fills the code in for them, which Android’s SMS Retriever and the
314 WebOTP API do without a keystroke [15, 16]. The design assumes that
315 for 20 % of verifications, at which five instant verifications in a row give a
316 ratio above 1 000, and the number must be set from the deployment’s own
317 measured distribution. The counters restart after each verdict.

318 What a verdict does is graded, because a wrong verdict on a block is
319 a wrong verdict on 10 000 numbers. The first verdict makes the block’s

320 first-time clients solve an interactive challenge for an hour; a second inside
 321 that hour moves them to non-SMS channels. Clients with verified history, a
 322 fingerprint or a number that verified before, are never affected. The 24-hour
 323 denylist is kept as an option; Section 7.5 measures what it buys. Finally, a
 324 carrier outage is not a pumper: the tests are suspended for fifteen minutes on
 325 a prefix whose delivery receipts collapse across at least ten of its blocks, or
 326 whose *returning* clients stop verifying across at least ten blocks. Only clients
 327 with verified history count toward the second signal, so an attacker cannot
 328 buy a suspension with a decoy flood without burning verified accounts to do
 329 it.

330 4.5. Capacity controls

331 Step 8 doubles the wait between messages to the same number, 60 s, 120 s,
 332 up to an hour, and caps a number at five messages a day. Step 9 enforces
 333 caps per source, platform and country, multiplied by a factor between 0.25
 334 and 1.5 that an hourly job sets from the key’s conversion and its deviation
 335 from a four-week baseline; a tightened cap never rations a client with verified
 336 history, and a request moved off SMS does not consume it. Step 10 watches
 337 the hourly message count and the hourly spend in prefix cost units against
 338 a budget, moves the pipeline to an elevated mode at 80 % and an emergency
 339 mode at 100 %, and pages on-call. In emergency mode the cleanest traffic
 340 still gets SMS, so that an attack cannot turn the breaker into a denial of
 341 service against real users; a manual kill switch stops all SMS.

342 Appendix Appendix A lists every default and the reason for it; the pseu-
 343 docode for each step is in the repository [17].

344 5. A model of pumping leakage

345 Against the block tests, a pumper’s leakage does not depend on how many
 346 numbers it uses but on how many blocks it touches. Let B be the number of
 347 8-digit blocks the pumper’s ranges intersect, λ its request rate, N its total
 348 requests, and k the number of sends a block costs before its verdict when
 349 every send goes the attacker’s way. For the conversion test k is the smallest
 350 integer with $k \ln \frac{1-p_a}{1-p_\ell} > \ln 1000$, where $p_\ell = 0.8$ and $p_a = 0.1$ are the assumed
 351 verification probabilities of a real user and a flooder; that gives $k = 5$. For
 352 the speed test, with $q_\ell = 0.2$ and $q_a = 0.9$ the probabilities of an instant
 353 verification, $k = \lceil \ln 1000 / \ln(q_a/q_\ell) \rceil = 5$ as well. Then

$$L \approx \min(N, kB + \lambda \tau), \quad (2)$$

where τ is the time a send stays unresolved: the resolution timeout plus the delivery delay plus half the period of the worker that resolves timeouts for a carrier that never verifies, about 2.5 minutes with our defaults, and only the delivery delay plus the carrier’s own entry delay for one that verifies at once.

A carrier that verifies a share σ of its codes with human-like delay moves the conversion test’s expected increment per send to $\sigma \ln \frac{p_a}{p_\ell} + (1 - \sigma) \ln \frac{1-p_a}{1-p_\ell}$, which is zero at

$$\sigma^* = \frac{\ln \frac{1-p_a}{1-p_\ell}}{\ln \frac{1-p_a}{1-p_\ell} - \ln \frac{p_a}{p_\ell}} = 0.42. \quad (3)$$

Above that share the test drifts away from a verdict. The price is that each verified code is a registered account on a number the pumper controls, so evasion costs the pumper at least σ^* fake accounts per message it is paid for, and the defender inherits them. Section 7.5 checks Eq. (2) against simulation at nine points and Eq. (3) against a carrier verifying 60 %.

6. Implementation and evaluation method

6.1. Implementation

The pipeline, its store, the feedback loop and the vendor adapters are about 3000 lines of Python, with one method per step. All state sits behind one store interface with an in-memory backend for tests and a Redis backend for deployment; every rate limit is one Lua script. Adapters exist for Google reCAPTCHA, Play Integrity, App Attest assertions, two IP-intelligence services, a number-lookup service, Twilio SMS and WhatsApp, push notifications and an alert sink, each with a stated policy on vendor outage: CAPTCHA and attestation fail closed, IP intelligence and number lookup fail open with a neutral result. A FastAPI service exposes the session, request, verify and delivery-receipt endpoints and a worker resolves timeouts and runs the adaptive baseline job. The test suite has 205 tests, each pipeline test on both backends. No live vendor call has been made from the repository; adapters are tested against recorded responses.

6.2. Simulation

Because no production logs were available to us, the evaluation is a minute-by-minute simulation of one registration source. Each run warms up with ten minutes of legitimate traffic, then runs twenty minutes of attack

with legitimate traffic in the background at 20 requests per minute. Legitimate users verify 80 % of codes after a lognormal delay (median 22s from delivery, typed; median 2s for the 20 % whose device fills the code in); 20 % of requests come from a browser that verified before; 5 % leave through an in-country hosting range, 5 % roam from abroad and 2 % sit behind a cloud proxy abroad. Delivery receipts arrive after a lognormal delay with median 3s. Prices are published list prices: 0.1422 USD per SMS to Saudi Arabia, 0.008 USD per number lookup, 0.001 USD per CAPTCHA assessment, 0.003 USD per solved CAPTCHA token, 3 USD per gigabyte of residential proxy traffic. Where a figure is an assumption rather than a published one, the repository’s calibration table says so; the legitimate conversion of 80 % is such an assumption, chosen above the 68 % that one large provider reports as its global average [18].

Each attacker runs on thirty seeds. Per seed, the pool size (500 to 50 000 addresses), the request rate (10 to 60 per minute) and the CAPTCHA score class are drawn at random. Results are means with 95 % *t*-intervals over seeds; the intervals cover seed-to-seed variation, not calibration error. Two modes are run: *behavioural only*, with the source caps lifted so that the other layers are visible, and *with source caps*, at three times the legitimate rate, adaptive for v2 and static for v1, since the adaptive multiplier is a v2 feature. v1 is the first design run through the same code with every v2 layer switched off by a feature flag.

6.3. Metrics

For the attacker: total messages leaked over the twenty minutes; time to containment, the first minute after which leakage never again exceeds 5 % of the attack rate; messages leaked before containment; steady-state leakage per minute after it; and the defender’s cost in messages, lookups and assessments. For real users: delivered, challenged and refused as a share of their requests, and the added delay. For the block tests: blocks that reached a verdict, and real users a verdict touched.

6.4. Attackers

7. Results

7.1. RQ1: what is stopped, and at what cost

Table 5 and Figure 1 answer the first question. Four attacks are stopped outright, with no message sent at any seed: the spoofed header, because

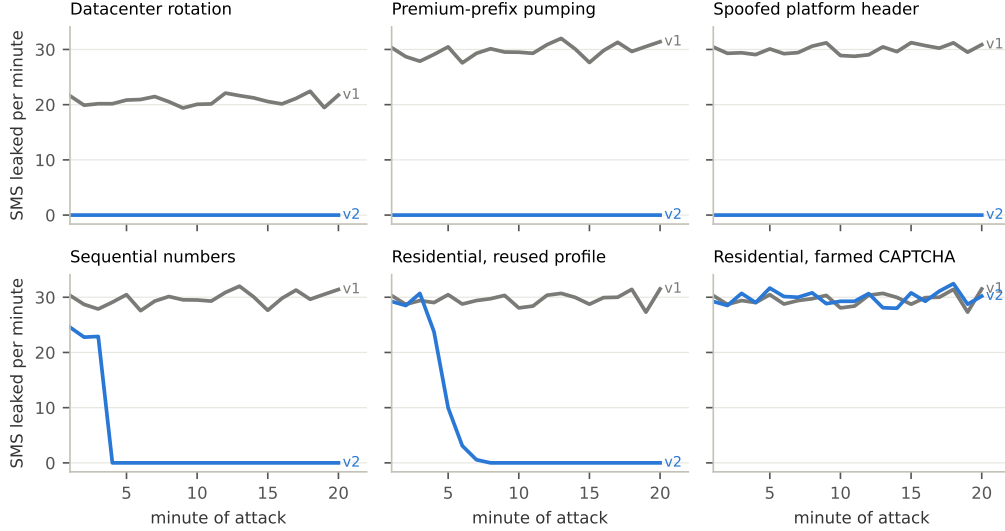


Figure 1: Messages leaked per minute of attack, v1 against v2, means over thirty seeds with the source caps lifted. The first three attacks never send a message under v2; the sequential walk and the reused profile are stopped within five minutes; the farmed-CAPTCHA attacker is untouched.

Step 0 derives the platform from attestation; datacenter rotation, because the hosting ASN and the fresh fingerprint put every request in the block tier; premium-prefix pumping, because Step 5 refuses the range; and, in all but a trickle, the single client, which the per-session cap holds to two messages. The sequential walk is stopped at minute 3.5 after 63 to 70 messages, because every number it tries sits in one destination block and that block reaches a verdict after five unverified sends; v1 leaked it whole. The reused browser profile is stopped at minute 4.7, because its fingerprint accumulates its own history. In every one of these cases 99 % of real users are delivered and about 2 % are challenged, the latter being the corporate, roaming and cloud-proxy minority whose addresses carry risk points.

The last two rows are the other half of the answer. A residential attacker with bot-grade CAPTCHA scores is held mostly by Step 3 under both designs. One with farmed scores, fresh or pre-aged fingerprints and valid numbers leaks everything with the caps lifted under both designs, 593 and 597 of 600, and the two intervals overlap. With the caps on, v1’s static cap leaks 563 and refuses 4 % of real users; v2’s adaptive cap leaks 241 and refuses 42 % of first-time users (returning users are exempt and are all delivered). All of

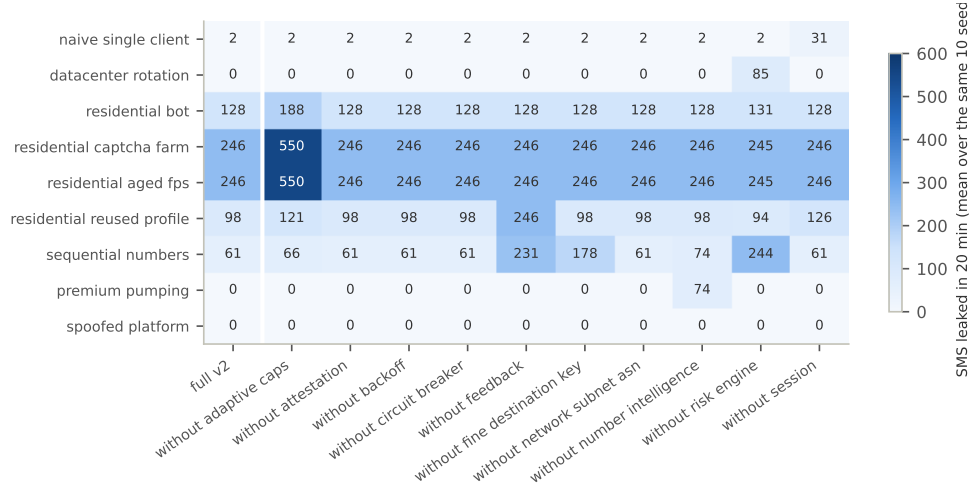


Figure 2: Leave-one-layer-out ablation with the source caps on. Each cell is the mean over the same ten seeds; the first column is the complete design on those seeds.

438 v2’s gain against this attacker is rationing. We return to why in Section 7.3.

439 7.2. RQ2: which layer does what

440 Figure 2 removes one layer at a time, with every column on the same ten
 441 seeds. Each recognisable attack has one layer that stops it: the session cap
 442 for the naive client (2 to 31 without it), the risk engine for datacenter rotation
 443 (0 to 85), number intelligence for premium pumping (0 to 74). The feedback
 444 loop stops the reused profile (98 to 246 without it) and, together with the
 445 destination-block key, the sequential walk (61 with both; 231 without the
 446 loop, 178 without the block key, 244 without the risk engine). Against the two
 447 farmed-CAPTCHA attackers only one column changes: without the adaptive
 448 cap they leak 550 instead of 246. Every other layer, the loop included, is
 449 invisible to them. An earlier version of this ablation compared a thirty-seed
 450 reference against ten-seed columns and concluded that the loop contributed
 451 nothing; that comparison mixed seed sets and was wrong, which is why the
 452 reference column here is computed on the same seeds.

453 7.3. RQ3: where reputation stops separating

454 The conversion penalty attaches to the keys a request touches. A residen-
 455 tial attacker with farmed scores touches an address it uses once, a fingerprint

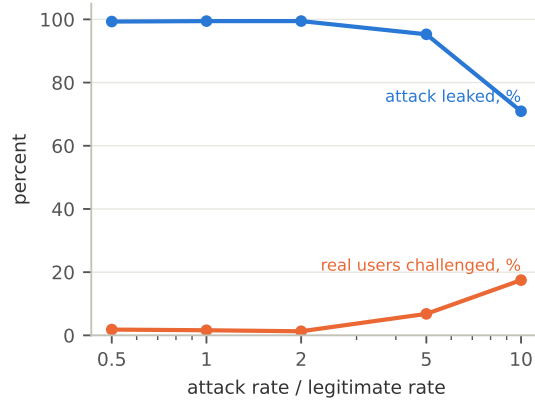


Figure 3: The farmed-CAPTCHA attacker at multiples of the legitimate rate for an hour, with the source caps lifted. The conversion penalty starts to fire near twice the legitimate volume and fires on real users as it does so.

456 it uses once, and a country, a prefix and a residential ASN it shares with ev-
 457 ery real user. Its failures therefore land on the shared keys, and the penalty
 458 they produce lands on everyone. That is rationing, not separation.

459 Two experiments show that no setting changes this. Figure 3 raises the
 460 attack from half the legitimate rate to ten times it for an hour. Up to twice
 461 the legitimate rate the attack leaks 99 %: the shared keys’ ratios stay above
 462 0.3 because real users keep verifying. At five times, 95 % leaks and 7 % of
 463 real users are challenged; at ten times, 71 % leaks and 18 % are challenged.
 464 The penalty starts near 1.8 times the legitimate volume, but starting is not
 465 separating. The parameter sweep (Table 6) says the same thing from the
 466 other side. Raising the conversion weight from 25 to 40, or cutting the OTP
 467 timeout to two minutes, changes nothing because the signal never fires on
 468 these keys. Scaling the tier boundaries to 0.6 cuts leakage by about 40 %
 469 and challenges 18 % of real users. Raising the fresh-fingerprint weight to 40
 470 stops the attacker outright and challenges 46 % of real users and refuses 10 %,
 471 because most real sign-ups are also new browsers.

472 The lesson generalises beyond this design. Any reputation built on out-
 473 comes separates an attacker from real users only on keys the attacker dom-
 474 inates, and the attacker decides how pure the client-side keys are: an ad-
 475 dress, a fingerprint and a session can be made single-use for nothing, and a
 476 CAPTCHA solution costs 0.3 cents. That leaves two kinds of key where sep-
 477 aration is possible. Keys the attacker reuses by mistake or for convenience,

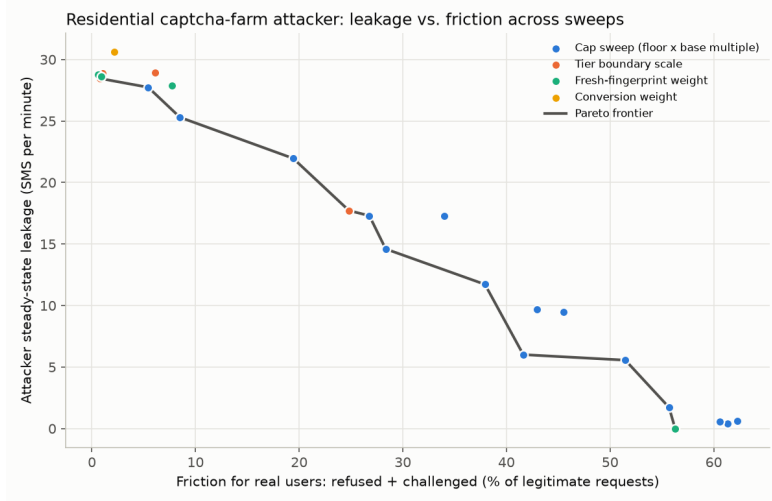


Figure 4: Steady-state leakage of the farmed-CAPTCHA attacker against friction for real users (refused plus challenged), over the cap and weight sweeps. The line is the Pareto frontier.

like the reused browser profile, and keys it cannot rotate at all.

7.4. RQ4: what rationing costs

When separation is impossible, the only lever is the volume cap, and Figure 4 measures what it costs. Along the frontier, holding the attacker to 10 messages a minute refuses about 40 % of first-time users; holding it to under one a minute refuses 61 %; letting it through refuses almost none. The adaptive multiplier’s floor and the base cap set where on this line a deployment sits (an adaptive floor of 0.25 at three times the legitimate rate, the default, leaks 246 and refuses 41 %), and the circuit breaker sets a ceiling on spend beyond which the line is left. There is no point on it that a service would call good; the honest statement is that against this attacker the design trades money for first-time sign-ups at a measured rate, and that returning users are kept out of the trade.

7.5. RQ3 continued: pumping on the key the attacker cannot rotate

Table 7 is the pumping result. A pumper whose carrier does not verify is contained in every seed by minute 5 with 104 messages leaked; one whose carrier enters every code within a second is caught by the speed test by minute 1.5 with 18 leaked, and it loses money at any revenue share we

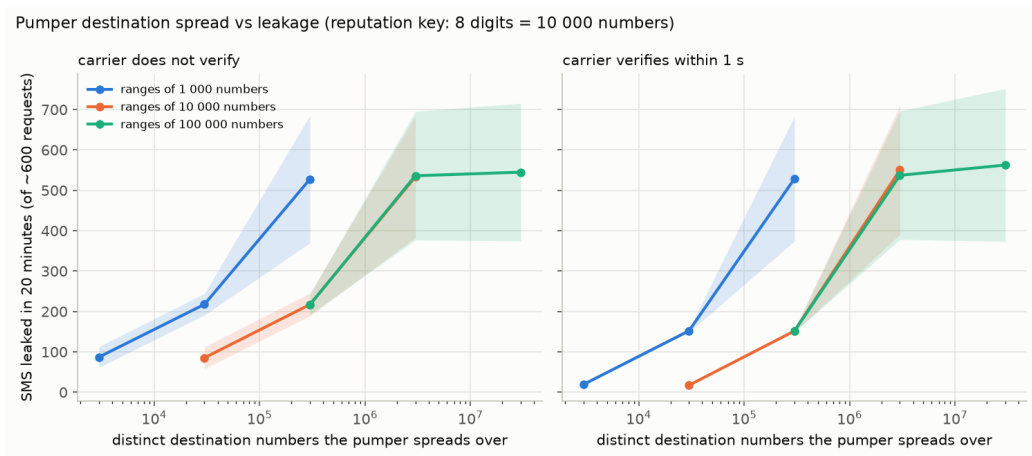


Figure 5: Messages leaked against the number of distinct destination numbers a pumper spreads over, with ranges that fit inside, align with, or span the 8-digit key. Leakage depends on the blocks touched, not the numbers used.

496 assumed. A pumper that buys a solution to every challenge earns a second
 497 verdict on its blocks and is moved off SMS, at 122 leaked against 104 under
 498 the hard denylist; that is what the denylist buys, eighteen messages, against a
 499 lockout of 10 000 numbers for a day. The floodor control is untouched by any
 500 of this, as it should be: it touches blocks at random and never concentrates
 501 on one.

502 A carrier that verifies 60 % of its codes after a human-like delay defeats
 503 every variant, leaks 559 of 600, and leaves 323 verified accounts on numbers
 504 it controls. Equation (3) said it needed 42 %; 60 % is comfortably past it.
 505 The pumping has not been stopped; its cost has moved from the SMS bill to
 506 whatever those accounts are later used for, and measuring it there is outside
 507 this paper.

508 Figure 5 and Table 8 vary how widely the pumper spreads. Ranges of
 509 1000 and 10 000 numbers behave identically because each sits in one block;
 510 ranges of 100 000 numbers touch ten blocks each and leak accordingly; and
 511 at 300 blocks the pumper is the floodor again. The model of Eq. (2) predicts
 512 every point to within a few messages: 88 against 85 to 87 measured at three
 513 blocks, 223 against 216 to 218 at thirty, and saturation at three hundred;
 514 for the instant verifier, 17 against 17 to 19, and 152 against 152. The in-
 515 flight term is what separates the two carriers at small B : the non-verifier's
 516 sends stay unresolved for the two-minute window and keep leaking while the

517 verdicts land, the verifier’s resolve within seconds.

518 *7.6. RQ5: what the block tests cost real users*

519 The block tests are the one part of the design that acts on a key real people
520 share by the thousand, and they were calibrated to the simulated population.
521 Three experiments ask what they do when the population differs, over a day,
522 and during an outage.

523 The first is a Monte Carlo of the exact tests on one block that sees only
524 legitimate traffic (Table 9). With the tests as deployed, assuming 80 % con-
525 version and 20 % fast verifications, a population that converts at 80 % pro-
526 duces a verdict on about one block in a thousand per day, and a 30 % autofill
527 share against the 20 % calibration adds under one percent. A population
528 that converts at 65 % produces a verdict on about 2 % of blocks with thirty
529 or more sends a day; one at 50 %, on 9 to 25 %. The conversion calibra-
530 tion, not the speed one, is the sensitive parameter, and it must come from
531 measured traffic.

532 The second runs the simulation for 24 hours with no attacker, with legit-
533 imate numbers drawn from a fixed set of blocks so that each block sees 144,
534 29 or about 4 sends a day. At 80 % conversion the day produces between
535 zero and two verdicts and touches at most one or two of 29 000 real users.
536 At 65 % it produces 3 to 18 verdicts and touches 1 to 22 users, who meet a
537 challenge rather than a refusal; delivery stays at 99.3 % in every cell. The
538 third stops one prefix, a seventh of the traffic, for thirty minutes inside an
539 hour, in two forms: the provider reports every send failed, or reports deliv-
540 ery while nobody receives anything. With receipts gating the tests and the
541 carrier-wide suspension, neither form produces a verdict, and an alert names
542 the carrier. Without the suspension the silent outage flags 1.4 blocks and
543 touches 1.8 users; the send-clocked design we started with flags 1.4 blocks in
544 both forms. These are the numbers that justified the receipt gating and the
545 graded verdict.

546 *7.7. Adaptive attackers, economics and overhead*

547 A colluding carrier on a range the prefix table lists as elevated-cost is
548 held to twenty messages by the per-prefix cap whether it verifies instantly or
549 with human-like delay; on an unlisted standard range the human-like one is
550 the pumper of Table 7. A datacenter attacker that buys a solution to every
551 challenge turns zero leaked messages into about fifty per run; the challenge

552 credit reduces friction for people and is not a defence against solvers. Low-
 553 and-slow attacks under the dilution bound leak in full at their own rate: 42
 554 messages in twenty minutes at two per minute over twenty ASNs, 199 at ten
 555 per minute.

556 With revenue credited only to pumpers, at a termination price of 0.1422
 557 USD and a revenue share of 20 to 50 %, premium-prefix pumping earns 14
 558 to 40 USD per twenty minutes under v1 and loses 1.78 USD under v2; the
 559 instant-verifying concentrated pumper loses money under v2; the human-like
 560 one earns 14 to 38 USD under both designs and adds 323 accounts.

561 Overhead was measured against a real Redis on a four-vCPU container
 562 with four worker processes and vendors faked: a full send costs 14.8 ms at the
 563 median and 27 ms at the 99th percentile in process, with 46 Redis round trips,
 564 and the service serves 203 requests per second at concurrency 32 without the
 565 response floor. Of those 46 round trips, five are the delivery receipt, the block-
 566 test counters and the outage windows. With the 400 ms floor, 6 000 requests
 567 across four outcomes showed no distinguishable pair by Kolmogorov–Smirnov
 568 test for five of six pairs; the sixth, a sent code against a repeated number,
 569 was marginal ($p = 0.04$, mean difference 0.4 ms), which at six tests is what
 570 chance produces and which an equivalence test within 2 ms rejects as a usable
 571 gap for every pair. Without the floor every pair is distinguishable.

572 8. Discussion: lessons and limitations

573 8.1. Lessons

- 574 1. *Reputation separates attackers only on keys they dominate.* Everything
 575 the evaluation found reduces to this. The attacker chooses the purity
 576 of the client-side keys and can make them single-use for nothing; the
 577 defender’s signal lands on whatever the attacker shares with real users.
 578 Tuning cannot change it; Table 6 tried.
- 579 2. *The destination is the key a pumper cannot rotate, and its cost is linear*
 580 *in blocks touched.* Equation (2) held at every point we measured. It
 581 also says what a pumper must do to evade: spread over hundreds of
 582 blocks, at which point it is paid on fewer of them, or verify 42 % of its
 583 codes, at which point it is manufacturing accounts.
- 584 3. *A wrong verdict on a shared key must be graded.* Ten thousand numbers
 585 are behind each block. A challenge for an hour, then non-SMS channels,
 586 with returning users exempt, cost eighteen messages against the hard
 587 denylist and avoided a day’s lockout.

- 588 4. *Failure is a delivery receipt and a timeout, not a timeout alone.* Clock-
589 ing from the send made a slow route look like a machine and an outage
590 look like a pumper. Clocking from the receipt fixed both and cost five
591 round trips.
- 592 5. *Calibration is part of the defence.* The block tests assume the deploy-
593 ment’s conversion and its autofill share. Set wrong, they flag real blocks
594 at a measurable rate; the Monte Carlo of Table 9 is the tool to check
595 them with.
- 596 6. *A request moved off SMS must not consume the SMS cap.* We found
597 this by watching real users refused on the attacker’s behalf, and the fix
598 is one line.

599 8.2. Limitations

600 Everything here is simulated. No logs from the incident or the v1 period
601 were available to us, so every absolute number is conditional on the calibra-
602 tion, and the calibration marks which of its figures are assumptions. The
603 repository contains a replay tool and a log schema so that the comparison
604 can be rerun on production traffic when it becomes available; that is the one
605 addition that would change what this paper can claim. The number-lookup
606 fake says every number is live, which makes the random-number attackers
607 harder than reality on that axis. The prefix table is a sample; an incomplete
608 one in production leaks premium ranges. Legitimate behaviour is stylised:
609 one request per user, no retries after a failed delivery, and every user on
610 the same ISP as the residential attacker, which is the worst case for dilu-
611 tion. The autofill share and the returning-user share are assumptions, and
612 the false-positive study sweeps the first because the speed test’s calibration
613 depends on it. The outage detector’s conversion signal needs ten returning
614 users’ failures, which on a low-traffic prefix takes most of half an hour; the
615 delivery-receipt signal is faster and only as honest as the provider’s reports.
616 A colluding carrier could fake failed receipts for other people’s sends on its
617 own prefix to suspend the tests there, at the price of an alert every window;
618 we did not model it. Attackers do not adapt within a run except through
619 the adaptive profiles. The confidence intervals cover seeds, not the model.

620 8.3. Ethics and privacy

621 The design fingerprints browsers, keeps addresses and phone numbers in
622 reputation counters for a day and in audit records for thirty, and challenges
623 people on the strength of what strangers on their network did. Each of those

624 is a choice with a cost, and a deployment in the European Union or in Saudi
625 Arabia will need a lawful basis, a retention statement and a way for a refused
626 user to reach a person. No human data was used in this work; the simulation's
627 users are synthetic. The incident is described with the operator's permission
628 and without identifying it.

629 **9. Conclusion**

630 An OTP endpoint is an SMS budget behind a form, and the first thing
631 that stops a flood is not the last. The design in this paper stops every attack
632 that can be recognised from the request itself without touching real users,
633 contains a pumper on the one key it cannot rotate at a cost that has a closed
634 form, and measures what it does to real people when its assumptions are
635 wrong. It does not separate a residential attacker that looks like a person,
636 and we have shown that no setting of its parameters will; what it offers there
637 is a measured trade between money and first-time sign-ups, with returning
638 users kept out of it. Those three results, one positive, one negative and one
639 bounded, are what a layered defence of this kind can honestly claim, and the
640 artifact lets anyone check them.

641 **Artifact availability**

642 The implementation, the evaluation harness, every recorded result and
643 the scripts that regenerate them are at [https://github.com/nahmedpsu/0](https://github.com/nahmedpsu/0TP-DDOS)
644 `TP-DDOS` (version 2.4.0; an archived DOI will be added on acceptance). `make`
645 `results` reproduces every table and figure in this paper from fixed seeds;
646 the full tables with confidence intervals are in `results/evaluation.md` and
647 `results/performance.md`.

648 **CRedit authorship contribution statement**

649 **Naveed Ahmed:** Conceptualization, Methodology, Software, Validation,
650 Investigation, Writing – original draft, Writing – review & editing.
651 **TODO:** add co-authors.

652 **Declaration of competing interest**

653 The authors declare that they have no known competing financial inter-
654 ests or personal relationships that could have appeared to influence the work
655 reported in this paper.

656 Declaration of generative AI and AI-assisted technologies in the 657 writing process

658 During the preparation of this work the author used Claude (Anthropic)
659 to assist with writing and reviewing code, running the evaluation, and draft-
660 ing and editing the manuscript text. The author reviewed and verified every
661 number, claim and reference, and takes full responsibility for the content of
662 the publication.

663 Data availability

664 No real traffic data was used. All synthetic data, seeds and results are in
665 the repository above.

666 Appendix A. Default values and the reasons for them

667 References

- 668 [1] MITRE Corporation, Resource hijacking: SMS pumping, technique
669 T1496.003, MITRE ATT&CK, <https://attack.mitre.org/techniques/T1496/003/>, accessed 1 October 2026.
- 671 [2] Commsrisk, Elon Musk says Twitter lost \$60mn a year because 390
672 telcos used bot accounts to pump A2P SMS, <https://commsrisk.com/elon-musk-says-twitter-lost-60mn-a-year-because-390-telcos-used-bot-accounts-to-pump-a2p-sms/>, trade press report of a
674 public statement; the figure is the company’s own claim (2023).
- 676 [3] J. H. Huh, H. Shin, S. Ahn, H. Yi, J. Cho, T. Kim, M. Lim, N. Choi,
677 Preventing artificially inflated SMS attacks through large-scale traffic in-
678 spection, in: 34th USENIX Security Symposium, USENIX Association,
679 Seattle, WA, 2025.
- 680 [4] M. Sahin, A. Francillon, P. Gupta, M. Ahamad, SoK: Fraud in telephony
681 networks, in: 2017 IEEE European Symposium on Security and Privacy
682 (EuroS&P), IEEE, 2017. doi:10.1109/EuroSP.2017.40.
- 683 [5] M. Sahin, A. Francillon, Understanding and detecting international rev-
684 enue share fraud, in: Network and Distributed System Security Sym-
685 posium (NDSS), Internet Society, 2021. doi:10.14722/ndss.2021.24051.

- 686 [6] Twilio Inc., Verify fraud guard, [https://www.twilio.com/docs/ver-](https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard)
687 [ify/preventing-toll-fraud/sms-fraud-guard](https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard), product documenta-
688 tion. Accessed 1 October 2026.
- 689 [7] M. Motoyama, K. Levchenko, C. Kanich, D. McCoy, G. M. Voelker,
690 S. Savage, Re: CAPTCHAs – understanding CAPTCHA-solving ser-
691 vices in an economic context, in: 19th USENIX Security Symposium,
692 USENIX Association, 2010.
- 693 [8] A. Dionysiou, E. Athanasopoulos, SoK: Machine vs. machine – A sys-
694 tematic classification of automated machine learning-based CAPTCHA
695 solvers, *Computers & Security* 97 (2020) 101947. doi:10.1016/j.cose
696 .2020.101947.
- 697 [9] X. Mi, X. Feng, X. Liao, B. Liu, X. Wang, F. Qian, Z. Li, S. Alrizah,
698 L. Chen, Y. Qian, Resident evil: Understanding residential IP proxy
699 as a dark service, in: 2019 IEEE Symposium on Security and Privacy
700 (S&P), IEEE, 2019. doi:10.1109/SP.2019.00011.
- 701 [10] P. Richter, F. Wolsing, A. Chandrasekaran, R. Padmanabhan, M. All-
702 man, R. Bush, A. Feldmann, A multi-perspective analysis of carrier-
703 grade NAT deployment, in: *Proceedings of the 2016 ACM Internet*
704 *Measurement Conference (IMC)*, ACM, 2016. doi:10.1145/298744
705 3.2987474.
- 706 [11] S. Wiefling, L. Lo Iacono, M. Dürmuth, Is this really you? An empirical
707 study on risk-based authentication applied in the wild, in: *ICT Systems*
708 *Security and Privacy Protection (IFIP SEC 2019)*, Vol. 562 of *IFIP Ad-*
709 *vances in Information and Communication Technology*, Springer, 2019,
710 pp. 134–148. doi:10.1007/978-3-030-22312-0_10.
- 711 [12] C. Mulliner, R. Borgaonkar, P. Stewin, J.-P. Seifert, SMS-based one-
712 time passwords: Attacks and defense, in: *Detection of Intrusions and*
713 *Malware, and Vulnerability Assessment (DIMVA 2013)*, Vol. 7967 of
714 *LNCS*, Springer, 2013, pp. 150–159. doi:10.1007/978-3-642-39235
715 -1_9.
- 716 [13] H. Siadati, T. Nguyen, P. Gupta, M. Jakobsson, N. Memon, Mind your
717 SMSes: Mitigating social engineering in second factor authentication,

- 718 Computers & Security 65 (2017) 14–28. doi:10.1016/j.cose.2016.
719 09.009.
- 720 [14] A. Wald, Sequential tests of statistical hypotheses, The Annals of Math-
721 ematical Statistics 16 (2) (1945) 117–186. doi:10.1214/aoms/1177731
722 118.
- 723 [15] Google LLC, SMS retriever API overview, <https://developers.google.com/identity/sms-retriever/overview>, developer documenta-
724 tion. Accessed 1 October 2026.
- 726 [16] Web Incubator Community Group, WebOTP API, <https://github.com/WICG/WebOTP>, specification repository. Accessed 1 October 2026.
727
- 728 [17] N. Ahmed, OTP Guard: a layered SMS validation, risk-scoring and
729 rate-limiting pipeline against OTP flood abuse (version 2.4.0), <https://github.com/nahmedpsu/OTP-DDOS>, source code, evaluation and
730 recorded results. Archived DOI to be added (2026).
731
- 732 [18] Twilio Inc., Verify API, <https://www.twilio.com/en-us/user-authentication-identity/verify>, product page stating a 68%+ global
733 conversion rate. Accessed 1 October 2026.
734

Table 2: The twelve steps and the gap each one closes. FB is the asynchronous feedback loop.

Step	Check	What it closes
0	Proxy block; platform derived from attestation, never from the header	Gap A
1	Signed session token bound to a fingerprint; nonce replay; 3 requests per session per 10 min	Gap B, replay
2	Atomic limits per address, per /24 subnet and per ASN, lower for hosting ASNs	Gaps B, F
3	reCAPTCHA v3 with a score threshold, web only	bots
4	Origin check, web only	domain allow-list
5	Number intelligence: E.164 parsing, country, prefix cost class, sequential and narrow-range detection, liveness lookup	Gaps C, D
6	Body length; excluded numbers	body limits
7	Risk score and tier: allow, delay, challenge, downgrade, block	Gap H
8	Per-number backoff: 60 s doubling to 1 h; five per day	per-number flooding
9	Per-source, per-platform, per-country caps with an adaptive multiplier	aggregate caps
10	Global circuit breaker on hourly count and spend	Gap E
11	Channel selection; audit log; uniform response body and a constant response time	Gap H
FB	Verification feedback: verified, failed, undelivered per key; destination-block tests; auto-denylist of fine keys	the signal an attacker cannot fake without phones

Table 3: Signals and weights in the risk score. Positive values raise risk.

Signal	Points
reCAPTCHA score c (web)	$(1 - c) \times 25$
Address in a hosting or datacenter ASN	+15
Address abuse-list score $a \in [0, 1]$	$a \times 15$
Fingerprint first seen under 5 min ago / under 1 h ago	+20 / +10
Each previous request in this session	+5
Lowest conversion ratio $r_{\min}$ over the request's keys, when below 0.3	up to +25
Sustained flood on a key (≥ 100 resolved sends, ratio < 0.1)	+15
Address country differs from number country	+10
Sequential number; narrow-range burst	+15 each
Elevated-cost prefix; unknown prefix; VoIP number	+10 each
App without attestation inside the grace window	+20, never better than delay
Interactive challenge solved	-20
Number verified before; fingerprint verified before	-20; -15

Table 4: Attacker profiles. All residential attackers draw addresses from in-country residential ranges; “farmed” CAPTCHA scores follow the human distribution.

Profile	Description
<i>Main study</i>	
naive single client	One address, one session, random numbers, a basic bot’s CAPTCHA scores
datacenter rotation	Hosting ASN abroad, fresh fingerprint per request, headless-browser scores
residential bot	Residential pool, fresh fingerprint, bot or headless scores
residential CAPTCHA farm	Residential pool, fresh fingerprint, farmed scores
residential aged fingerprints	Residential pool, unique fingerprints seeded two hours earlier, farmed scores
residential reused profile	Residential pool, one 48-hour-old browser profile reused
sequential numbers	Residential pool, numbers walked upward
premium pumping	Residential pool, a premium-rate prefix
spoofed platform	Residential pool, the iOS header without attestation
<i>Adaptive</i>	
pumper, listed range	Colluding carrier on an elevated-cost range, verifies every code within 1 s, or 60 % after 30 s
pumper, unlisted range	As above, on a range the prefix table does not know
challenge solver	Datacenter rotation that buys a solution to every interactive challenge
low and slow	2 or 10 requests per minute over 20 residential ASNs, aged fingerprints
<i>Pumping study</i>	
concentrated pumper	Three 10 000-number blocks inside a standard prefix; carrier does not verify, verifies within 1 s, verifies 60 % after 30 s, or buys challenge solutions

Table 5: Messages leaked in twenty minutes (about 600 requests) and the share of real users refused, means over thirty seeds. Full tables with intervals are in the repository.

Attacker	caps lifted		with source caps: leaked / refused	
	v1	v2	v1 (static)	v2 (adaptive)
Naive single client	31	2	31 / 1 %	2 / 1 %
Datacenter rotation	416	0	400 / 2 %	0 / 1 %
Premium-prefix pumping	595	0	558 / 4 %	0 / 1 %
Spoofed platform header	599	0	100 / 0 %	0 / 1 %
Sequential numbers	595	70	558 / 4 %	63 / 2 %
Residential, reused profile	593	126	563 / 4 %	93 / 6 %
Residential, bot CAPTCHA scores	219	221	219 / 1 %	160 / 12 %
Residential, farmed CAPTCHA	593	597	563 / 4 %	241 / 42 %

Table 6: Weight and boundary sweep against the farmed-CAPTCHA attacker, caps lifted, ten seeds. The default is 25 for the conversion weight, 20 for the fresh-fingerprint weight, 1.0 for the boundary scale and 120 s for the resolution timeout.

Parameter	Value	Leaked / 600	Real users challenged	Real users refused
default	—	586	1.4 %	0.8 %
conversion weight	40	586	1.4 %	0.8 %
resolution timeout	120 s $\rightarrow$ 600 s	586	1.4 %	0.8 %
boundary scale	0.6	357	18.2 %	6.6 %
fresh-fingerprint weight	40	0	46.0 %	10.2 %

Table 7: Concentrated pumpers on three 10 000-number blocks, caps lifted, ten seeds; messages leaked of about 600, time to containment, and verified fake accounts the carrier created. Default: block key, two-minute resolution, graded verdicts.

Carrier	Design	Leaked	Contained at (min)	Fake accounts
does not verify	v1	563	never	0
	v2 default	104	4.9	0
	v2, 24-h	104	4.9	0
	denylist			
verifies within 1 s	v1	562	never	562
	v2 default	18	1.5	18
verifies 60 % after 30 s	v1	563	never	331
	v2 default	559	never	323
does not verify; buys challenge solutions	v2 default	122	5.7 ^a	0
	v2, 24-h	104	4.9	0
	denylist			
farmed-CAPTCHA flooder (control)	v2 default	576	never	0

^a Contained in nine of ten seeds.

Table 8: The leakage model against the spread sweep (ten seeds). B is the number of 8-digit blocks the pumper’s ranges touch.

Carrier	Ranges $\times$ size	B	Measured	Model
does not verify	$3 \times 1\,000; 3 \times 10\,000$	3	87; 85	88
	$3 \times 100\,000; 30 \times 1\,000; 30 \times 10\,000$	30	216; 218; 217	223
	$30 \times 100\,000; 300 \times 1\,000; 300 \times 10\,000$	300	536; 528; 533	$\approx N$
verifies within 1 s	$3 \times 1\,000; 3 \times 10\,000$	3	19; 17	17
	$3 \times 100\,000; 30 \times 1\,000; 30 \times 10\,000$	30	152; 152; 152	152
	$30 \times 100\,000; 300 \times 1\,000; 300 \times 10\,000$	300	537; 529; 550	$\approx N$

Table 9: Probability that a block seeing only legitimate traffic reaches a verdict in one day, by true conversion and true share of verifications within 5 s of delivery; 30 sends per block per day; tests calibrated to 80 % and 20 %. Monte Carlo, 20 000 trials per cell.

Conversion	fast 0 %	5 %	10 %	20 %	30 %
50 %	18.5 %	18.6 %	18.6 %	18.4 %	19.1 %
65 %	1.9 %	1.9 %	1.8 %	1.8 %	2.6 %
80 %	0.03 %	0.10 %	0.07 %	0.11 %	0.69 %
90 %	0.00 %	0.01 %	0.02 %	0.07 %	0.78 %

Setting	Default	Reason
Tier boundaries	20 / 40 / 60 / 80	A clean web user scores under 15; one strong signal alone lands in <i>delay</i> , never <i>block</i> ; refusing needs three independent signals
Conversion penalty	below 0.3, at least 20 resolved sends	Legitimate conversion runs well above 50 %; under 30 % is not a bad day, it is a flood; 20 resolved sends keeps one user's typos from triggering it
Resolution timeout	120 s after delivery	99 % of simulated real users verify within about 100 s; a late verification is reclassified
Auto-denylist	ratio below 0.1 on 50 resolved sends, 24 h; fine keys and hosting ASNs only	No legitimate population converts under 10 % on fifty sends; a residential ASN is thousands of people
Block tests	likelihood ratio 1 000; $p_\ell = 0.8$, $p_a = 0.1$; $q_\ell = 0.2$, $q_a = 0.9$; 5 s from receipt	False-positive probability 0.2 ⁵ per block at the calibration; q_ℓ must be measured
Block verdict	challenge for 1 h, then non-SMS; verified clients exempt	Bounds a wrong verdict to first-time users for an hour
Outage suspension	15 min; 10 blocks; half of 20 sends undelivered, or returning users under 30 % on 10 sends in 30 min	Only returning users count for the conversion signal, so a decoy flood cannot buy it
Per-address cap	5 / min; 30 / min for listed carrier-NAT ASNs	Carriers put hundreds of subscribers behind one address; at 5 / min half a normal sign-up flow is refused
Circuit breaker	elevated at 80 %, emergency at 100 % of an hourly budget set to twice the p99 legitimate hour	Room for the elevated mode to act before the hard cap
Response floor	400 ms	Above the pipeline's wall-time p99 with one worker per vCPU; it must be sized per deployment
Per-number daily cap; session cap	5; 3 per 10 min	A real user retries two or three times; three covers a resend with one to spare